【扔物线 Compose 课程讲义】4.1-状态转移型动画 animateXxxAsState()

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：https://rengwuxian.com/cp

用法

Kotlin | 复制代码

```kotlin
val size by animateDpAsState(if (big) 96.dp else 48.dp)
// 也可以这么写：
val size by animateDpAsState(targetSize) // 然后通过修改 targetSize 来实现动画
```

关键点在于，参数里的数值是可变的，而在数值改变之后，`animateXxxAsSize()` 左边的值不会立即改变，而是会渐变，从而实现动画效果。

重点

- `animateXxxAsState()` 包含了自动刷新（Recompose）功能和对 `remember {}` 的嵌套，所以不用再额外写 `mutableStateOf()`，也不用额外写 `remember {}`；
- 由于 `animateXxxAsState()` 返回的是 `State` 对象而不是 `MutableState`，所以 `by` 的左边应该写 `val` 而不是 `var`；
  - 那想修改属性的值怎么办？——课上讲过了，直接去修改传入的参数，而不是拿着 `animateXxxAsState()` 返回的对象去修改。
- 由于 `animateXxxAsState()` 内部包含了 `remember {}` 的调用，所以它是一个 Composable 函数，所以不能在 Composable 环境的外部使用。

扔物线学堂 - 秉心 (1437182)